Barrett Viator
DSE6211
Professor Salemi
February 16, 2025

*ABC Hotels Reservation Retention Risk Preliminary Results Report*

Developing an accurate Feed Forward Neural Network for Deep Learning requires the appropriate parameters, activation functions, loss functions, and optimization algorithms to be selected which is a task that involves comparing outputs of multiple models to solidify the certainty behind the predictions being made. Without the proper preparation of the data and the careful fine-tuning of the aforementioned aspects, business decisions can not be made in a truly data-driven method. The preliminary results of the deep learning model have provided the insights that it is possible to find trends within the data that support the efforts of targeting consumers with specific behaviors in order to prevent the amount of reservation cancellations that ABC Hotels is currently experiencing.

In order to produce an accurate preliminary model, three layers were ultimately selected at this stage for the model after encountering overfitting quickly with the initial attempts that included more layers for a deeper network. After testing multiple numbers of units in each layer, the best results were obtained with the most amount of comparable epochs reached by using 200 units in the primary layer and transitioning to 100 units for the output layer. ReLU and Sigmoid activation functions were ultimately selected for this part of the binary classification problem due to their ability to work well with the loss function method of BCEloss. RMSprop was selected for the optimization algorithm due to its ability to perform more robustly in comparison to SGD and Adam. A loop was also introduced during the training process to halt the training cycle early when the training loss increased for more than a period of 15 epochs in order to prevent overtraining the model and compromising final outputs.

After several iterations with the parameters in the neural network needing to be adjusted due to overfitting, it appears that the model is performing at a rate that is starting to accurately predict the label in order for business recommandons to be made. The model returned an 88.49% accuracy when computing the ROC AUC Score which allows for confidence in the ability of the preliminary model to perform well. Additionally the scatter plot for this model shows a decent amount of clustering of each predicted class along the respective lines of accurate classification. Further adjustments of the neural network parameters and explorations of other activation functions need to be assessed in order to see if the accuracy of the current model can be improved or if the current findings can be used as a final model for extracting feature

importance. By working with other parameters and comparing several ROC AUC Scores, more certainty can be reached to select a final model to be used in practice. This will allow for business decisions to be made by the appropriate teams based on the current data available to ABC Hotels and with future data that is collected after enhanced consumer-targeting has been undertaken.

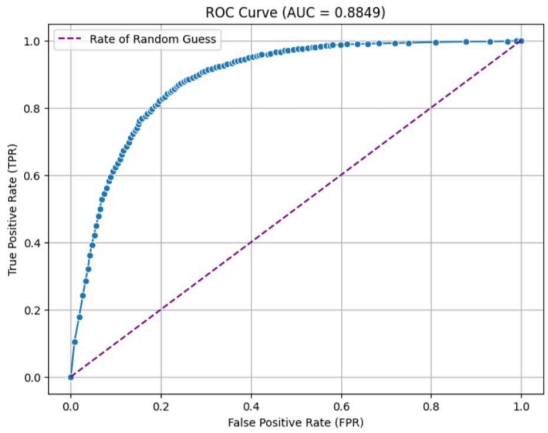



Fig 1. Preliminary Results ROC Curve

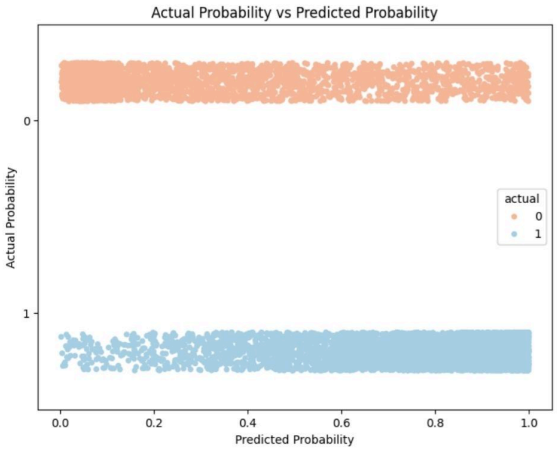



Fig 2. Preliminary Results Scatter Plot

A variety of options are available to produce deep learning models as the project moves towards producing final insights. Leaky RELU has the potential in future models to address the issue of "dying neurons" which could allow for deeper training with more epochs to confirm or enhance the findings in the preliminary results. Tanh is another activation function that is worth exploring even though it is not as efficient time wise and has the issue of the "vanishing gradient" to be mindful of when analyzing the results. Comparing multiple models with different activation functions gives the potential to make predictions that solidify the validity of other

models. Without comparing the current model to other models that may not be as efficient with their output, determining the level of confidence in the final model is not as sound.

Optimization algorithms and loss functions are additional parameters that can be adjusted when applying to the varying activation functions that help build confidence in the best performing model. Adam performs better than SGD which makes it an excellent choice for producing a comparison model. Hinge loss as a loss function is also worth exploring in future models because it penalizes incorrect predictions. BCEWithLogutsLoss as a loss function improves upon issues with floating-point precision. Finally, using Permutation Importance with the model selected that performs the best will allow for the features that impact the neural network the most to be ranked and extrapolated upon by those who will ultimately make the decisions of how to allocate resources to prevent future clients deemed at risk for reservation cancellation.

Given the ability of the preliminary model to perform well, ABC Hotels can be more certain that the right insights will be produced in the final report with a high level of accuracy. Enhancing these findings by comparing them to the outputs of other models ultimately provides the teams working to improve outcomes for both the consumers and ABC Hotels with necessary guidance to ensure funds are allocated in the right direction. By taking advantage of both the data available to ABC Hotels and the most current technological advancements, ABC Hotels can secure a more profitable future than if these decisions were made using more traditional methods.

# Preliminary Results Appendix

February 16, 2025

[3]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import auc
import torch
import torch.nn as nn
import torch.optim as optim
import seaborn as sns
import matplotlib.pyplot as plt
```

[4]:
```
data = pd.read_csv('C:\Program Files\Merrimack\DSE6211\Project\project_data.
 ↪csv')
df = data.drop('Booking_ID', axis=1)
```

[5]:
```
X = df.drop('booking_status', axis=1)
y = df['booking_status']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25,␣
 ↪random_state=123)
```

[6]:
```
encoder = OneHotEncoder(handle_unknown='ignore')
encoder.fit(X_train[['market_segment_type', 'lead_time', 'type_of_meal_plan',␣
 ↪'room_type_reserved', 'arrival_date',
                     'no_of_adults', 'no_of_children', 'no_of_weekend_nights',␣
 ↪'no_of_week_nights',
                    'required_car_parking_space', 'repeated_guest',␣
 ↪'no_of_previous_cancellations',
                    'no_of_previous_bookings_not_canceled',␣
 ↪'avg_price_per_room', 'no_of_special_requests']])
encoded_df = pd.DataFrame(encoder.transform(X_train[['market_segment_type',␣
 ↪'lead_time', 'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
                     'no_of_adults', 'no_of_children', 'no_of_weekend_nights',␣
 ↪'no_of_week_nights',
                    'required_car_parking_space', 'repeated_guest',␣
 ↪'no_of_previous_cancellations',
```

```
                    'no_of_previous_bookings_not_canceled',␣
 ↪'avg_price_per_room', 'no_of_special_requests']]).toarray(),
                           columns=encoder.
 ↪get_feature_names_out(['market_segment_type', 'lead_time',␣
 ↪'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
                     'no_of_adults', 'no_of_children', 'no_of_weekend_nights',␣
 ↪'no_of_week_nights',
                    'required_car_parking_space', 'repeated_guest',␣
 ↪'no_of_previous_cancellations',
                    'no_of_previous_bookings_not_canceled',␣
 ↪'avg_price_per_room', 'no_of_special_requests']))
X_train = pd.concat([X_train.drop(['market_segment_type', 'lead_time',␣
 ↪'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
                     'no_of_adults', 'no_of_children', 'no_of_weekend_nights',␣
 ↪'no_of_week_nights',
                    'required_car_parking_space', 'repeated_guest',␣
 ↪'no_of_previous_cancellations',
                    'no_of_previous_bookings_not_canceled',␣
 ↪'avg_price_per_room', 'no_of_special_requests'], axis=1).
 ↪reset_index(drop=True), encoded_df], axis=1)
X_train.columns
```

[6]:
```
Index(['market_segment_type_aviation', 'market_segment_type_complementary',
       'market_segment_type_corporate', 'market_segment_type_offline',
       'market_segment_type_online', 'lead_time_0', 'lead_time_1',
       'lead_time_2', 'lead_time_3', 'lead_time_4',
       …
       'avg_price_per_room_349.63', 'avg_price_per_room_365.0',
       'avg_price_per_room_375.5', 'avg_price_per_room_540.0',
       'no_of_special_requests_0', 'no_of_special_requests_1',
       'no_of_special_requests_2', 'no_of_special_requests_3',
       'no_of_special_requests_4', 'no_of_special_requests_5'],
      dtype='object', length=4373)
```

[7]:
```
scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns)
```

[8]:
```
X_train_scaled.head()
```

[8]:

| | market_segment_type_aviation | market_segment_type_complementary | \ |
|---|---|---|---|
| 0 | -0.056669 | -0.105648 | |
| 1 | -0.056669 | -0.105648 | |
| 2 | -0.056669 | -0.105648 | |
| 3 | -0.056669 | -0.105648 | |
| 4 | -0.056669 | -0.105648 | |

```
   market_segment_type_corporate  market_segment_type_offline  \
0                      -0.243564                     1.564504
1                      -0.243564                    -0.639180
2                      -0.243564                     1.564504
3                      -0.243564                     1.564504
4                      -0.243564                    -0.639180

   market_segment_type_online  lead_time_0  lead_time_1  lead_time_2  \
0                   -1.332491    -0.192487    -0.174822    -0.135922
1                    0.750474    -0.192487    -0.174822     7.357159
2                   -1.332491    -0.192487    -0.174822    -0.135922
3                   -1.332491    -0.192487    -0.174822    -0.135922
4                    0.750474    -0.192487    -0.174822    -0.135922

   lead_time_3  lead_time_4  …  avg_price_per_room_349.63  \
0    -0.134795    -0.135218  …                  -0.006066
1    -0.134795    -0.135218  …                  -0.006066
2    -0.134795    -0.135218  …                  -0.006066
3    -0.134795    -0.135218  …                  -0.006066
4    -0.134795    -0.135218  …                  -0.006066

   avg_price_per_room_365.0  avg_price_per_room_375.5  \
0                 -0.006066                 -0.006066
1                 -0.006066                 -0.006066
2                 -0.006066                 -0.006066
3                 -0.006066                 -0.006066
4                 -0.006066                 -0.006066

   avg_price_per_room_540.0  no_of_special_requests_0  \
0                 -0.006066                  0.914048
1                 -0.006066                 -1.094035
2                 -0.006066                 -1.094035
3                 -0.006066                  0.914048
4                 -0.006066                 -1.094035

   no_of_special_requests_1  no_of_special_requests_2  \
0                 -0.675059                 -0.370986
1                  1.481353                 -0.370986
2                  1.481353                 -0.370986
3                 -0.675059                 -0.370986
4                  1.481353                 -0.370986

   no_of_special_requests_3  no_of_special_requests_4  \
0                 -0.138151                 -0.047038
1                 -0.138151                 -0.047038
2                 -0.138151                 -0.047038
```

```
3                  -0.138151                  -0.047038
4                  -0.138151                  -0.047038

   no_of_special_requests_5
0                   -0.01486
1                   -0.01486
2                   -0.01486
3                   -0.01486
4                   -0.01486

[5 rows x 4373 columns]
```

[9]:
```
X_train_tensor = torch.tensor(X_train_scaled.values, dtype=torch.float32)
```

[10]:
```
X_train_tensor
```

[10]:
```
tensor([[-0.0567, -0.1056, -0.2436,  …, -0.1382, -0.0470, -0.0149],
        [-0.0567, -0.1056, -0.2436,  …, -0.1382, -0.0470, -0.0149],
        [-0.0567, -0.1056, -0.2436,  …, -0.1382, -0.0470, -0.0149],
        …,
        [-0.0567, -0.1056, -0.2436,  …, -0.1382, -0.0470, -0.0149],
        [-0.0567, -0.1056, -0.2436,  …, -0.1382, -0.0470, -0.0149],
        [-0.0567, -0.1056,  4.1057,  …, -0.1382, -0.0470, -0.0149]])
```

[11]:
```
encoded_df = pd.DataFrame(encoder.transform(X_test[['market_segment_type',␣
 ↪'lead_time', 'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
                     'no_of_adults', 'no_of_children', 'no_of_weekend_nights',␣
 ↪'no_of_week_nights',
                    'required_car_parking_space', 'repeated_guest',␣
 ↪'no_of_previous_cancellations',
                    'no_of_previous_bookings_not_canceled',␣
 ↪'avg_price_per_room', 'no_of_special_requests']]).toarray(),
                           columns=encoder.
 ↪get_feature_names_out(['market_segment_type', 'lead_time',␣
 ↪'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
                     'no_of_adults', 'no_of_children', 'no_of_weekend_nights',␣
 ↪'no_of_week_nights',
                    'required_car_parking_space', 'repeated_guest',␣
 ↪'no_of_previous_cancellations',
                    'no_of_previous_bookings_not_canceled',␣
 ↪'avg_price_per_room', 'no_of_special_requests']))
X_test = pd.concat([X_test.drop(['market_segment_type', 'lead_time',␣
 ↪'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
                     'no_of_adults', 'no_of_children', 'no_of_weekend_nights',␣
 ↪'no_of_week_nights',
```

```
                    'required_car_parking_space', 'repeated_guest',␣
 ↪'no_of_previous_cancellations',
                    'no_of_previous_bookings_not_canceled',␣
 ↪'avg_price_per_room', 'no_of_special_requests'], axis=1).
 ↪reset_index(drop=True), encoded_df], axis=1)
X_test.columns
```

[11]:
```
Index(['market_segment_type_aviation', 'market_segment_type_complementary',
       'market_segment_type_corporate', 'market_segment_type_offline',
       'market_segment_type_online', 'lead_time_0', 'lead_time_1',
       'lead_time_2', 'lead_time_3', 'lead_time_4',
       …
       'avg_price_per_room_349.63', 'avg_price_per_room_365.0',
       'avg_price_per_room_375.5', 'avg_price_per_room_540.0',
       'no_of_special_requests_0', 'no_of_special_requests_1',
       'no_of_special_requests_2', 'no_of_special_requests_3',
       'no_of_special_requests_4', 'no_of_special_requests_5'],
      dtype='object', length=4373)
```

[12]:
```
scaler = StandardScaler()
scaler.fit(X_test)
X_test_scaled = scaler.transform(X_test)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_train.columns)
```

[13]:
```
X_test_tensor = torch.tensor(X_test_scaled.values)
X_test_tensor.size()
```

[13]:
```
torch.Size([9060, 4373])
```

[14]:
```
encoder = LabelEncoder()
y_train_encoded = encoder.fit_transform(y_train)
y_train_tensor = torch.tensor(y_train_encoded, dtype=torch.float32)
y_test_encoded = encoder.fit_transform(y_test)
y_test_tensor = torch.tensor(y_test_encoded, dtype=torch.float32)
```

[15]:
```
y_train_tensor
```

[15]:
```
tensor([1., 1., 0.,  …, 1., 1., 0.])
```

[16]:
```
y_test_tensor
```

[16]:
```
tensor([1., 1., 1.,  …, 1., 0., 1.])
```

[17]:
```
y_test_np = y_test_tensor.numpy().reshape(-1, 1)
scaler = StandardScaler()
scaler.fit(y_test_np)
y_test_scaled = scaler.transform(y_test_np)
```

[18]:
```
X_test_np = X_test_scaled.to_numpy().astype(np.float32)
X_test_tensor = torch.tensor(X_test_np, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test_scaled, dtype=torch.float32)
```

[19]:
```
n_samples = X_train_tensor.shape[0]
n_val = int(0.2 * n_samples)
shuffled_indices = torch.randperm(n_samples)
train_indices = shuffled_indices[:-n_val]
val_indices = shuffled_indices[-n_val:]
train_indices, val_indices

X_tr = X_train_tensor[train_indices]
y_tr = y_train_tensor[train_indices]
X_val = X_train_tensor[val_indices]
y_val = y_train_tensor[val_indices]
```

[20]:
```
seq_model = nn.Sequential(
    nn.Linear(4373, 200),
    nn.ReLU(),
    nn.Linear(200, 100),
    nn.ReLU(),
    nn.Linear(100, 1),
    nn.Sigmoid()
)
```

[21]:
```
def training_loop(n_epochs, optimizer, model, loss_fn, X_tr, X_val, y_tr,␣
 ↪y_val, patience=15):
    best_val_loss = float("inf")
    counter = 0

    for epoch in range(1, n_epochs + 1):
        seq_model.train()

        X_tr_p = model(X_tr)
        loss_train = loss_fn(X_tr_p, y_tr.unsqueeze(1))

        optimizer.zero_grad()
        loss_train.backward()
        optimizer.step()

        seq_model.eval()
        with torch.no_grad():
            X_val_p = model(X_val)
            loss_val = loss_fn(X_val_p, y_val.unsqueeze(1))


        if epoch == 1 or epoch % 10 == 0:
```

```
            print(f"Epoch {epoch}, Training loss {loss_train.item():.4f},␣
↪Validation loss {loss_val.item():.4f}")

        if loss_val.item() < best_val_loss:
            best_val_loss = loss_val.item()
            counter = 0
        else:
            counter += 1

        if counter >= patience:
            print(f"Overfitting detected at epoch {epoch}. Training concluded.")
            print(f"Final Epoch: {epoch}, Training Loss: {loss_train.item():.
↪4f}, Validation Loss: {loss_val.item():.4f}")
            break
```

[22]:
```
optimizer = optim.RMSprop(seq_model.parameters(), lr=1e-4)
training_loop(
    n_epochs = 200,
    optimizer = optimizer,
    model = seq_model,
    loss_fn = nn.BCELoss(),
    X_tr = X_tr,
    X_val = X_val,
    y_tr = y_tr,
    y_val = y_val)
```

```
Epoch 1, Training loss 0.7125, Validation loss 0.6757
Epoch 10, Training loss 0.4981, Validation loss 0.5556
Epoch 20, Training loss 0.3860, Validation loss 0.4976
Epoch 30, Training loss 0.3205, Validation loss 0.4720
Epoch 40, Training loss 0.2781, Validation loss 0.4610
Epoch 50, Training loss 0.2470, Validation loss 0.4563
Epoch 60, Training loss 0.2217, Validation loss 0.4543
Epoch 70, Training loss 0.2000, Validation loss 0.4537
Epoch 80, Training loss 0.1808, Validation loss 0.4540
Early stopping triggered at epoch 87. Training stopped.
Final Epoch: 87, Training Loss: 0.1685, Validation Loss: 0.4547
```

[23]:
```
seq_model(X_test_tensor).detach().numpy()
```

[23]:
```
array([[0.78787166],
       [0.9671501 ],
       [0.9661977 ],
       …,
       [0.94713265],
       [0.0328461 ],
       [0.65751475]], shape=(9060, 1), dtype=float32)
```

[24]:
```
p_1_actual = pd.DataFrame({'p_1': seq_model(X_test_tensor).detach().numpy().
  ↪reshape(-1), 'actual': y_test_encoded})
p_1_actual.head()
```

[24]:

| | p_1 | actual |
|---|---|---|
| 0 | 0.787872 | 1 |
| 1 | 0.967150 | 1 |
| 2 | 0.966198 | 1 |
| 3 | 0.759832 | 1 |
| 4 | 0.988461 | 1 |

[25]:
```
roc_list = []

actuals_0 = (p_1_actual['actual'] == 0).sum()
actuals_1 = (p_1_actual['actual'] == 1).sum()

for i in np.arange(1, -0.01, -0.01):

    over_threshold = p_1_actual[p_1_actual['p_1'] >= i]

    fp = (over_threshold['actual'] == 0).sum()
    tp = (over_threshold['actual'] == 1).sum()

    fpr = fp / actuals_0 if actuals_0 > 0 else 0.0
    tpr = tp / actuals_1 if actuals_1 > 0 else 0.0

    roc_list.append((i, fpr, tpr))

roc_data = pd.DataFrame(roc_list, columns=['threshold', 'fpr', 'tpr'])

roc_data
```

[25]:

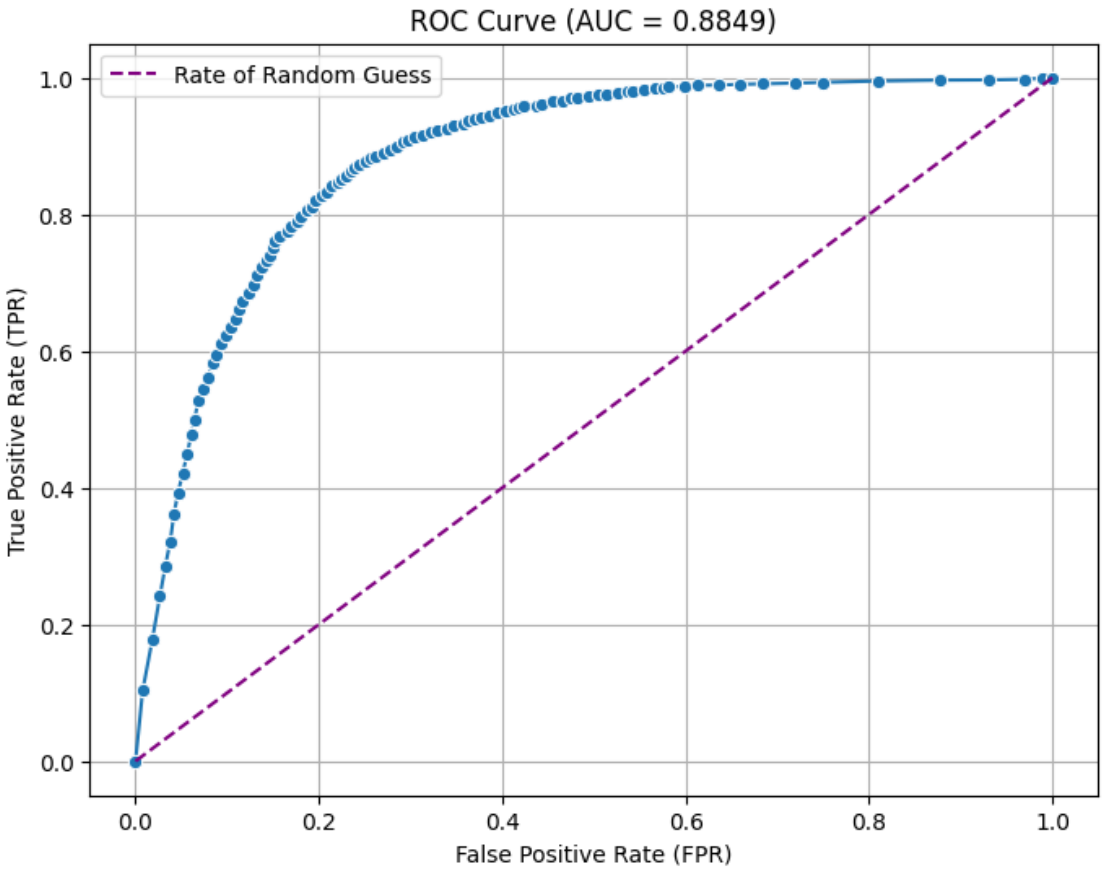
| | threshold | fpr | tpr |
|---|---|---|---|
| 0 | 1.000000e+00 | 0.000000 | 0.000000 |
| 1 | 9.900000e-01 | 0.007853 | 0.103572 |
| 2 | 9.800000e-01 | 0.018436 | 0.178601 |
| 3 | 9.700000e-01 | 0.026630 | 0.241722 |
| 4 | 9.600000e-01 | 0.032434 | 0.284782 |
| .. | … | … | … |
| 96 | 4.000000e-02 | 0.878115 | 0.996575 |
| 97 | 3.000000e-02 | 0.930352 | 0.996901 |
| 98 | 2.000000e-02 | 0.969273 | 0.997717 |
| 99 | 1.000000e-02 | 0.989075 | 0.999348 |
| 100 | -8.881784e-16 | 1.000000 | 1.000000 |

[101 rows x 3 columns]

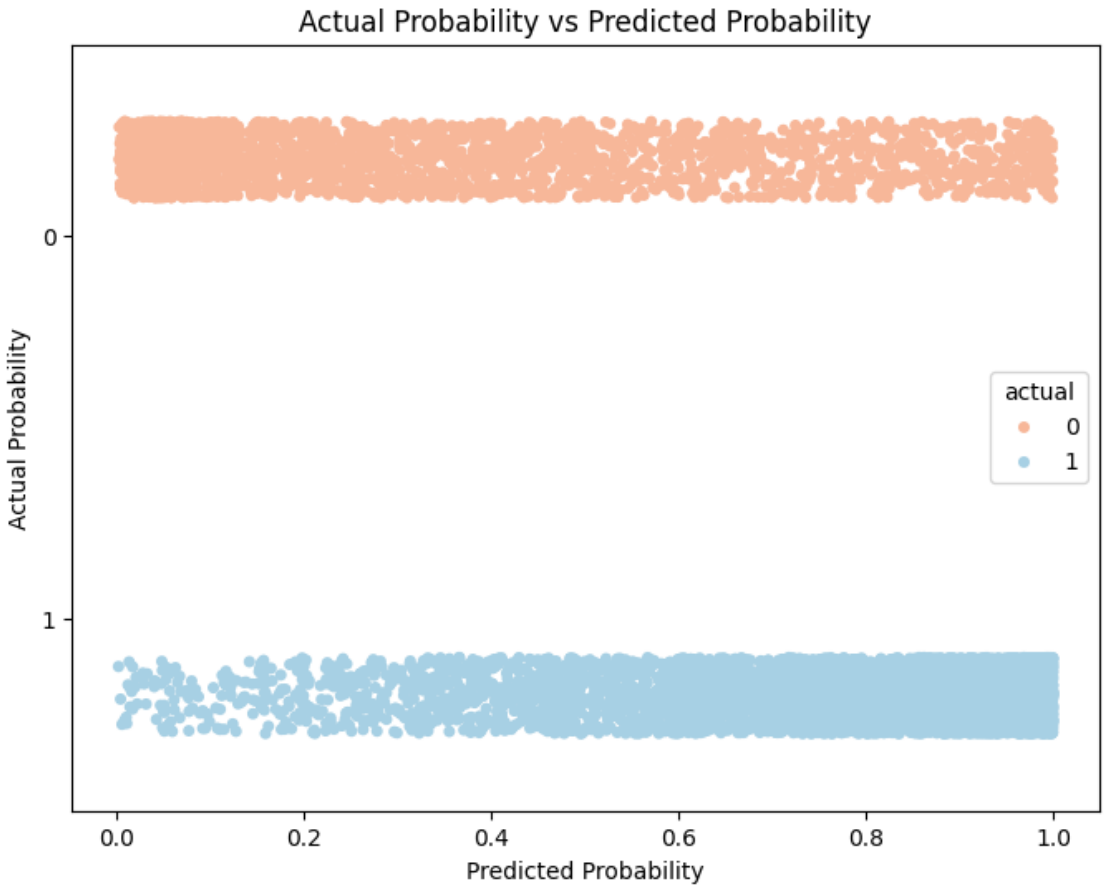
[26]:
```
roc_auc = auc(roc_data['fpr'], roc_data['tpr'])
print(f"ROC AUC Score: {roc_auc:.4f}")
```

```
ROC AUC Score: 0.8849
```

[27]:
```
plt.figure(figsize=(8, 6))
roc = sns.lineplot(x='fpr', y='tpr', data=roc_data, marker='o', linestyle='-')

xlims = roc.get_xlim()
ylims = roc.get_ylim()

plt.xlim(-0.05, 1.05)
plt.ylim(-0.05, 1.05)

plt.plot([0, 1], [0, 1], linestyle='--', color='purple', label="Rate of Random␣
↪Guess")

plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title(f'ROC Curve (AUC = {roc_auc:.4f})')
plt.legend()
plt.grid()

plt.show()
```




[28]:
```
p_1_actual['actual'] = p_1_actual['actual'].astype('category')

plt.figure(figsize=(8,6))
strip = sns.stripplot(x='p_1', y='actual', data=p_1_actual, jitter=0.2,␣
 ↪hue='actual', palette='RdBu', dodge=True)

plt.title('Actual Probability vs Predicted Probability')
plt.xlabel('Predicted Probability')
plt.ylabel('Actual Probability')

plt.show()
```


Actual Probability vs Predicted Probability
Actual Probability
0
1
Predicted Probability
0.0
0.2
0.4
0.6
0.8
1.0
actual
0
1

[ ]: